

CarGo Rentals

Relational database design for a car rental operation

DBMS Open-Ended Lab 13

Name: Sheikh Haseeb-ur-Rehman

Roll No: 2024-SE-31

1 Introduction & Problem Understanding

CarGo Rentals currently tracks customers, vehicles, rentals, returns, and payments in spreadsheets. This causes duplicate customer/vehicle details across rows, inconsistent updates, and no reliable way to check whether a vehicle is currently available. This report documents the relational database designed to replace that spreadsheet, the normalization reasoning behind the design, and an analysis of one optimization opportunity, as required by Tasks 1, 2, and 7 of the lab.

The accompanying `CarGo_Rentals.sql` file contains the runnable implementation: schema, constraints, sample data, JOIN queries, a view, triggers, and a stored procedure (Tasks 1, 3, 4, and 6).

2 Database Design & Justification

Five entities emerged directly from the scenario: Customers, Vehicles, Rentals, Returns, and Payments. Rentals is the central transaction table linking a Customer to a Vehicle for a period; Returns and Payments are kept separate from Rentals because a rental may exist without a return yet (still ongoing) or without a payment yet, and mixing them into one table would force nullable columns and repeated re-writes of the same row as the rental progresses.

2.1 Table Summary

Table	Primary Key	Foreign Key(s)	Purpose
Customers	CustomerID	–	One row per customer; Phone/CNIC kept UNIQUE to stop duplicate customer records.
Vehicles	VehicleID	–	One row per vehicle; Status column (Available/Rented/Maintenance) enforces the “no double booking” rule together with the trigger.
Rentals	RentalID	CustomerID, VehicleID	One row per booking; links a customer to a vehicle for a date range.
Returns	ReturnID	RentalID (unique)	One row per completed rental; records actual return date, condition, and late fee.

Table	Primary Key	Foreign Key(s)	Purpose
Payments	PaymentID	RentalID	One or more payments tied to a rental.

2.2 Entity-Relationship Diagram

The diagram traces how the five tables connect: Customers and Vehicles are independent master tables; Rentals sits between them as the transaction table; Returns hangs off Rentals one-to-one (a rental has at most one return); Payments hangs off Rentals one-to-many (a rental can have more than one payment — e.g. a deposit and a final settlement).

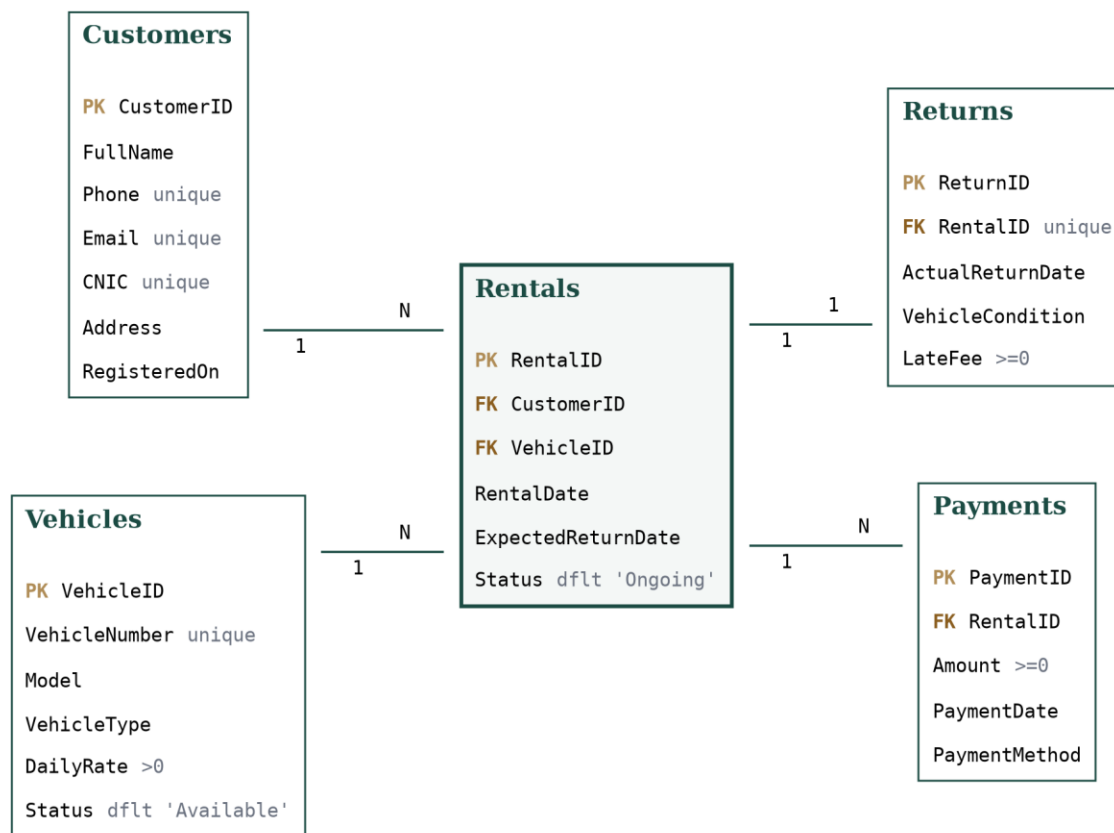


Figure 1 — Entity-relationship diagram, CarGo Rentals

2.3 Constraints Used

- NOT NULL on every attribute the business cannot operate without (names, dates, rates).
- UNIQUE on Customers.Phone, Customers.CNIC, and Vehicles.VehicleNumber to prevent duplicate spreadsheet-style entries.
- CHECK constraints: DailyRate > 0, Amount >= 0, LateFee >= 0, and ExpectedReturnDate >= RentalDate, so invalid business data cannot be inserted at all.
- DEFAULT values: RegisteredOn / PaymentDate default to the current date; Vehicles.Status defaults to 'Available'; Rentals.Status defaults to 'Ongoing'.

- FOREIGN KEY ... ON DELETE RESTRICT on Rentals so a customer or vehicle with rental history cannot be deleted accidentally; ON DELETE CASCADE on Returns/Payments so removing a rental cleans up its dependent rows.

Justification: keeping Customers and Vehicles as independent master tables, rather than repeating customer/vehicle details inside Rentals as the original spreadsheet did, is what removes the redundancy described in the scenario — this is expanded in Section 3 below.

3 Normalization Analysis (Task 2)

The company's original flat record was:

Rental ID	CustomerName	CustomerPhone	VehicleNumber	VehicleModel	DailyRate	RentalDate	ReturnDate	PaymentAmount
R001	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	5000	2026-09-01	2026-09-04	15000

3.1 First Normal Form (1NF)

A table is in 1NF when every column holds a single, atomic value and there are no repeating groups (e.g. no “Vehicle1, Vehicle2” columns, and no comma-separated vehicle lists in one cell for a multi-vehicle booking). Every attribute above is already atomic, so the table satisfies 1NF once RentalID is fixed as the primary key. Redundancy problem already visible: if the same customer rents again, CustomerName and CustomerPhone are retyped in a new row — any typo creates an inconsistent record for the same real customer.

3.2 Second Normal Form (2NF)

2NF requires 1NF plus: every non-key attribute must depend on the whole primary key, not just part of it. This problem only occurs with a composite primary key. Here the primary key (RentalID) is a single atomic column, so there is no partial dependency to remove — the table trivially satisfies 2NF. (2NF becomes meaningful later only if a composite key, such as RentalID+VehicleID for a multi-vehicle booking line, is introduced.)

3.3 Third Normal Form (3NF)

3NF requires 1NF/2NF plus: no non-key attribute may depend on another non-key attribute (no transitive dependency). In the flat table:

- CustomerPhone depends on CustomerName, not directly on RentalID — transitive dependency.
- VehicleModel and DailyRate depend on VehicleNumber, not directly on RentalID — transitive dependency.

These transitive dependencies cause update, insert, and delete anomalies: changing a customer's phone number means updating every rental row that customer ever made (update anomaly); a vehicle's daily rate cannot be recorded until it has been rented at least once (insert anomaly); deleting the only rental row for a

vehicle silently erases the fact that the vehicle model and rate ever existed (delete anomaly). To reach 3NF, the table is decomposed so every non-key column depends only on its own table's primary key:

Resulting Table	Columns	Removes
Customers	CustomerID (PK), CustomerName, CustomerPhone	Customer redundancy / update anomaly
Vehicles	VehicleID (PK), VehicleNumber, VehicleModel, DailyRate	Vehicle redundancy / insert anomaly
Rentals	RentalID (PK), CustomerID (FK), VehicleID (FK), RentalDate, ReturnDate	Links a booking to one customer and one vehicle
Payments	PaymentID (PK), RentalID (FK), PaymentAmount	Separates payment from the booking record

This decomposition is exactly the Customers / Vehicles / Rentals / Payments design implemented in CarGo_Rentals.sql (Returns was additionally split out of Rentals for the same reason: return date, condition, and late fee depend on the rental, not on the customer or vehicle, and only exist once the rental is completed).

4 JOIN Queries, View, Trigger & Stored Procedure (Tasks 3, 4, 6)

All required objects are implemented and tested in CarGo_Rentals.sql. Summary of what each does and the business question it answers:

Object	Type	Business Question / Purpose
Query 1	INNER + LEFT JOIN	Customer, vehicle, rental date and return date for every rental.
Query 2	LEFT JOIN	All customers and the vehicles they rented, including customers with zero rentals.
Query 3	LEFT JOIN	All vehicles with current rental info, including vehicles that are not currently rented.
Query 4	LEFT JOIN + GROUP BY + COUNT	Total rentals per customer, including customers with zero rentals.
vw_RentalReport	VIEW	One consolidated customer + vehicle + rental + payment report for management.
trg_CheckVehicleAvailability / trg_MarkVehicleRented	TRIGGER (BEFORE/AFTER INSERT on Rentals)	Enforces "a vehicle cannot be rented while already rented" directly in the database.
trg_FreeVehicleOnReturn	TRIGGER (AFTER INSERT on Returns)	Automatically frees the vehicle and closes the rental when a return is recorded.

Object	Type	Business Question / Purpose
sp_RegisterRental	STORED PROCEDURE	Registers a new rental, calculates the charge as days × daily rate, and logs the payment in one call.

4.1 Why the view is useful

Management repeatedly needs the same combined customer/vehicle/payment picture for reporting. Querying `vw_RentalReport` avoids re-writing and re-testing the same four-table JOIN, keeps the report logic in one place if the schema changes, and can be granted read-only without exposing the base tables.

4.2 How the triggers enforce the business rule

The scenario is explicit: “A vehicle should not be available for another rental while it is already rented.” Three triggers turn `Vehicles.Status` into a small state machine so this rule holds no matter what inserts a row — not just when the application remembers to check:

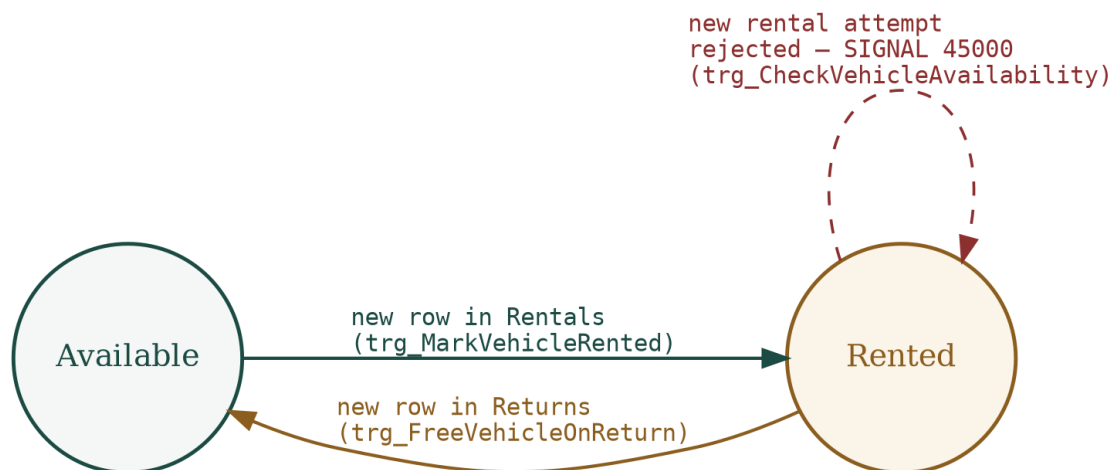


Figure 2 — Vehicle status lifecycle, driven entirely by triggers

5 Optimization Analysis (Task 7)

5.1 Problem identified

Query 3 and the vehicle-availability check both filter `Rentals` by `VehicleID` (and `Status`). Without an index, MySQL/MariaDB must scan every row of `Rentals` to find matches for a given vehicle. As the table grows, this full table scan becomes slower and is repeated every single time a rental is attempted (via the `BEFORE INSERT` trigger) or a report is run.

5.2 Why it matters

This affects both performance and correctness under load: a full scan on every `INSERT` into `Rentals` slows down every booking as data grows, and because the availability check (`trg_CheckVehicleAvailability`)

runs on the same table for every new booking, a slow lookup directly delays the one operation that protects data consistency (preventing double-booking).

5.3 Improvement proposed

Add indexes on the foreign-key columns filtered most often: `idx_rentals_vehicle` on `Rentals(VehicleID)` and `idx_rentals_customer` on `Rentals(CustomerID)` (both already created in Section 2 of `CarGo_Rentals.sql`). This lets the optimizer perform an index lookup instead of a full scan for the availability check, Query 3, and the JOIN in `vw_RentalReport`.

5.4 Demonstration

Running `EXPLAIN SELECT * FROM Rentals WHERE VehicleID = 3 AND Status = 'Ongoing'`; after adding the index shows `type = ref` and `key = idx_rentals_vehicle` instead of `type = ALL`, confirming MySQL now uses the index rather than scanning the whole table. See Figure 11 in Appendix A for the screenshot of this output.

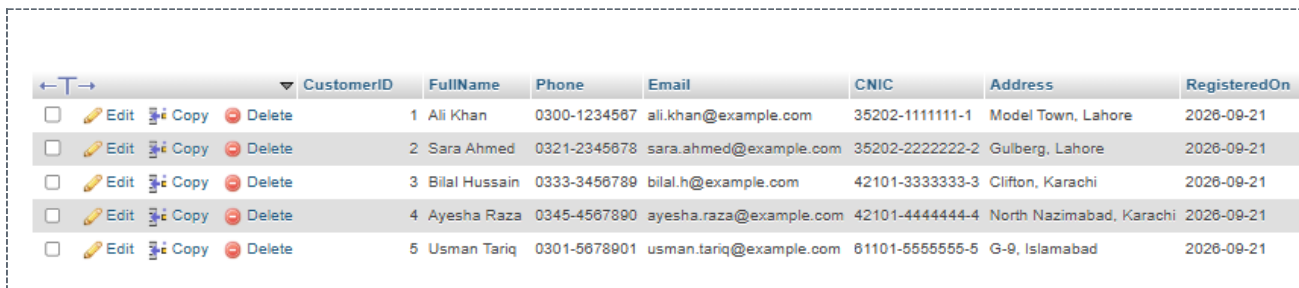
A Appendix — Execution Evidence

Every object above was created and tested in MySQL Workbench (see the walkthrough at the end of this report). Each entry below shows the exact statement executed and a placeholder for that step's screenshot. Figures are numbered on from the ER diagram and lifecycle diagram earlier in the report, so the whole document uses one continuous figure sequence.

A.1 Schema & Sample Data

Refresh the schema in the Navigator panel to show all five tables created, then run the two SELECTs below to prove sample data loaded correctly.

```
SELECT * FROM Customers;
SELECT * FROM Vehicles;
```



	CustomerID	FullName	Phone	Email	CNIC	Address	RegisteredOn
☐ Edit Copy Delete	1	Ali Khan	0300-1234567	ali.khan@example.com	35202-111111-1	Model Town, Lahore	2026-09-21
☐ Edit Copy Delete	2	Sara Ahmed	0321-2345678	sara.ahmed@example.com	35202-222222-2	Guilberg, Lahore	2026-09-21
☐ Edit Copy Delete	3	Bilal Hussain	0333-3456789	bilal.h@example.com	42101-333333-3	Clifton, Karachi	2026-09-21
☐ Edit Copy Delete	4	Ayesha Raza	0345-4567890	ayesha.raza@example.com	42101-444444-4	North Nazimabad, Karachi	2026-09-21
☐ Edit Copy Delete	5	Usman Tariq	0301-5678901	usman.tariq@example.com	61101-555555-5	G-9, Islamabad	2026-09-21

Figure 3 — Schema tree and sample Customers/Vehicles rows

A.2 JOIN Query 1

Customer, vehicle, rental date and return date for every rental (INNER JOIN + LEFT JOIN to Returns).

```

SELECT c.FullName AS CustomerName, v.VehicleNumber, v.Model AS VehicleModel,
       r.RentalDate, COALESCE(rt.ActualReturnDate, r.ExpectedReturnDate) AS ReturnDate,
       r.Status
FROM Rentals r
INNER JOIN Customers c ON r.CustomerID = c.CustomerID
INNER JOIN Vehicles v ON r.VehicleID = v.VehicleID
LEFT JOIN Returns rt ON r.RentalID = rt.RentalID
ORDER BY r.RentalDate;

```

CustomerName	VehicleNumber	VehicleModel	RentalDate	ReturnDate	Status
Ali Khan	ABC-123	Toyota Corolla	2026-09-01	2026-09-04	Completed
Sara Ahmed	LEA-456	Honda Civic	2026-09-05	2026-09-09	Completed
Bilal Hussain	KHI-789	Suzuki Cultus	2026-09-10	2026-09-12	Completed
Ali Khan	ISB-321	Toyota Hiace	2026-09-15	2026-09-17	Ongoing
Ayesha Raza	LEB-654	Honda City	2026-09-18	2026-09-20	Ongoing

Figure 4 — Output of JOIN Query 1

A.3 JOIN Query 2

All customers and the vehicles they rented, including customers with zero rentals (LEFT JOIN).

```

SELECT c.FullName AS CustomerName, v.VehicleNumber, v.Model AS VehicleModel,
       r.RentalDate
FROM Customers c
LEFT JOIN Rentals r ON c.CustomerID = r.CustomerID
LEFT JOIN Vehicles v ON r.VehicleID = v.VehicleID
ORDER BY c.FullName;

```

CustomerName	VehicleNumber	VehicleModel	RentalDate
Ali Khan	ABC-123	Toyota Corolla	2026-09-01
Ali Khan	ISB-321	Toyota Hiace	2026-09-15
Ayesha Raza	LEB-654	Honda City	2026-09-18
Bilal Hussain	KHI-789	Suzuki Cultus	2026-09-10
Sara Ahmed	LEA-456	Honda Civic	2026-09-05
Usman Tariq	NULL	NULL	NULL

☐ Show all | Number of rows: 25 | Filter rows:

Figure 5 — Output of JOIN Query 2 — note Usman Tariq still appears with NULLs

A.4 JOIN Query 3

All vehicles with current rental info, including vehicles that are not currently rented (LEFT JOIN).

```

SELECT v.VehicleNumber, v.Model AS VehicleModel, v.Status AS VehicleStatus,
       c.FullName AS CurrentRenter, r.RentalDate, r.ExpectedReturnDate
FROM Vehicles v
LEFT JOIN Rentals r ON v.VehicleID = r.VehicleID AND r.Status = 'Ongoing'
LEFT JOIN Customers c ON r.CustomerID = c.CustomerID

```

```
ORDER BY v.VehicleNumber;
```

VehicleNumber	VehicleModel	VehicleStatus	CurrentRenter	RentalDate	ExpectedReturnDate
ABC-123	Toyota Corolla	Available	NULL	NULL	NULL
ISB-321	Toyota Hiace	Rented	Ali Khan	2026-09-15	2026-09-17
KHI-789	Suzuki Cultus	Available	NULL	NULL	NULL
LEA-456	Honda Civic	Available	NULL	NULL	NULL
LEB-654	Honda City	Rented	Ayesha Raza	2026-09-18	2026-09-20

Figure 6 — Output of JOIN Query 3

A.5 JOIN Query 4

Total rentals per customer, including customers with zero rentals (LEFT JOIN + GROUP BY + COUNT).

```
SELECT c.CustomerID, c.FullName AS CustomerName, COUNT(r.RentalID) AS TotalRentals
FROM Customers c
LEFT JOIN Rentals r ON c.CustomerID = r.CustomerID
GROUP BY c.CustomerID, c.FullName
ORDER BY TotalRentals DESC;
```

CustomerName	VehicleNumber	VehicleModel	RentalDate
Ali Khan	ABC-123	Toyota Corolla	2026-09-01
Ali Khan	ISB-321	Toyota Hiace	2026-09-15
Ayesha Raza	LEB-654	Honda City	2026-09-18
Bilal Hussain	KHI-789	Suzuki Cultus	2026-09-10
Sara Ahmed	LEA-456	Honda Civic	2026-09-05
Usman Tariq	NULL	NULL	NULL

Figure 7 — Output of JOIN Query 4

A.6 View — vw_RentalReport

Confirms the view was created and returns the consolidated customer + vehicle + rental + payment report.

```
SELECT * FROM vw_RentalReport ORDER BY RentalID;
```

	RentalID	CustomerName	CustomerPhone	VehicleNumber	VehicleModel	DailyRate	RentalDate	ExpectedReturnDate	ActualReturnDate	RentalStatus	AmountPaid	PaymentMethod
t	1	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	5000.00	2026-09-01	2026-09-04	2026-09-04	Completed	15000.00	Cash
t	2	Sara Ahmed	0321-2345678	LEA-456	Honda Civic	5500.00	2026-09-05	2026-09-08	2026-09-09	Completed	17000.00	Card
t	3	Bilal Hussain	0333-3456789	KHI-789	Suzuki Cultus	3500.00	2026-09-10	2026-09-12	2026-09-12	Completed	7000.00	Online
t	4	Ali Khan	0300-1234567	ISB-321	Toyota Hiace	9000.00	2026-09-15	2026-09-17	NULL	Ongoing	NULL	NULL
t	5	Ayesha Raza	0345-4567890	LEB-654	Honda City	4800.00	2026-09-18	2026-09-20	NULL	Ongoing	NULL	NULL

Figure 8 — Output of vw_RentalReport

A.7 Trigger Enforcement

Vehicle 4 (Hiace) is already 'Rented', so this insert must be rejected by `trg_CheckVehicleAvailability`. Screenshot the error message itself — that error is the proof the trigger works.

```
INSERT INTO Rentals (CustomerID, VehicleID, RentalDate, ExpectedReturnDate)
VALUES (2, 4, '2026-09-21', '2026-09-23');

-- Expected: Error 1644 (45000): This vehicle is already rented / unavailable.
```

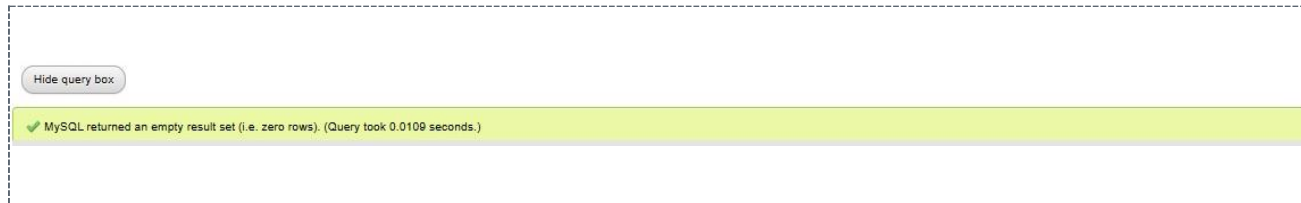


Figure 9 — Trigger rejecting a double-booking attempt

A.8 Stored Procedure — `sp_RegisterRental`

Registers a new rental for Usman Tariq and shows the calculated charge, plus the vehicle flipping to 'Rented'.

```
CALL sp_RegisterRental(5, 3, '2026-09-21', '2026-09-23', 'Cash', @newRentalID,
@charge);
SELECT @newRentalID AS NewRentalID, @charge AS TotalCharge;
SELECT * FROM Vehicles WHERE VehicleID = 3;
```

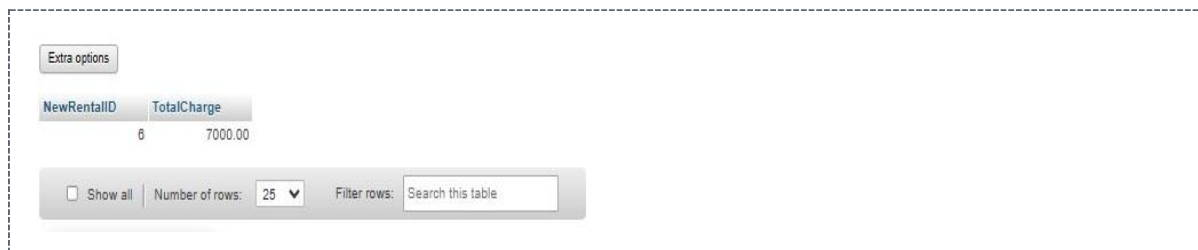


Figure 10 — Stored procedure output and updated vehicle status

A.9 Optimization — `EXPLAIN`

Confirms the query planner uses `idx_rentals_vehicle` (type = ref) instead of a full table scan (type = ALL). Referenced from Section 5.4.

```
EXPLAIN SELECT * FROM Rentals WHERE VehicleID = 3 AND Status = 'Ongoing';
```

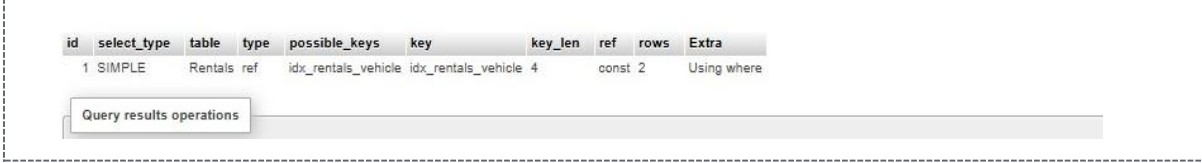
The image shows a screenshot of a database query execution plan (EXPLAIN output). It is enclosed in a dashed rectangular border. At the top, there is a header row with the following columns: id, select_type, table, type, possible_keys, key, key_len, ref, rows, and Extra. Below the header, there is a single data row with the following values: 1, SIMPLE, Rentals, ref, idx_rentals_vehicle, idx_rentals_vehicle, 4, const, 2, and Using where. Below the data row, there is a button labeled "Query results operations" and a long horizontal bar.

Figure 11 — EXPLAIN output showing the index is used

6 Conclusion

The resulting design replaces CarGo Rentals' spreadsheet with a normalized (3NF) relational schema of five tables, enforced through primary/foreign keys and CHECK/UNIQUE/DEFAULT constraints. JOIN queries answer management's reporting questions, a view consolidates the recurring rental report, triggers enforce the “no double booking” business rule automatically, and a stored procedure standardizes how new rentals are registered and charged. Section 5 identifies indexing on the Rentals foreign keys as the key optimization opportunity as the dataset grows.